EXPERIMENT 10

Applying Performance Optimization Techniques (Indexing, Subforms and Query Tuning) to a Zoho Creator Application

Application: Patient Appointment Scheduling System | Zoho Creator

Saranya V. | Register No: 2116241501188 | AIML | Rajalakshmi Engineering College

Abstract

This experiment adapts performance optimization techniques to the Patient Appointment Scheduling System in Zoho Creator. The report discusses typed fields, subforms and query tuning, and compares a loop-based count of pending appointments with a database-filtered count. The actual screenshots supplied for this project show the Appointment Booking Form, its report, the dashboard design and an existing Deluge workflow used to calculate net payable fees.

1. Aim

To understand and apply performance optimization techniques such as typed fields, subforms, filtered queries and reusable Deluge functions to the Patient Appointment Scheduling System.

2. Introduction

Performance optimization helps a low-code application remain responsive as its records grow. In this project, appointments are stored through Appointment Booking Form, displayed in a report, and summarized in a dashboard. The existing Auto-Calculate Net Fee workflow already uses Deluge. Similar care can be taken with report criteria and appointment-counting logic to avoid retrieving unnecessary records.

3. Concepts

1. Typed fields and indexing

Use appropriate field types for values that are frequently filtered. For example, Preferred Appointment Date should be a Date field, Appointment Status should be a choice/drop-down field if present, and Consultation Fee should be a numeric or currency field. Zoho Creator manages underlying storage/indexing; keep field types consistent and use report filters on the correct fields.

2. Subforms

A subform can store multiple related rows inside one parent record. For this project, it could be useful only if one appointment needs a repeating set of related details, such as multiple prescribed services or billing line items. Do not add a subform merely for optimization if the current appointment record has no repeating child data.

3. Query tuning

Filter records in the query itself and use count or aggregate operations rather than fetching every record and counting in a Deluge loop. Limit the fetched range when only a page of results is needed, and avoid fetching records inside a loop.

4. Reusable Deluge functions

The Workflow area includes a Functions tab where reusable Deluge logic can be stored. A function for counting pending appointments or calculating a summary can keep logic consistent when called from multiple workflows or pages.

4. Procedure

1. Open the Patient Appointment Scheduling System in Zoho Creator and select Workflow.

2. Open the Functions tab. If permitted in your account, click New Function and create a Deluge function for counting pending appointments.

3. First review the loop-based example below. It fetches all appointment records and checks the status one by one.

4. Create or draft the tuned version, which filters Appointment_Status == "Pending" in the query and uses count() to count matching records.

5. Confirm the actual form link name, status field link name and exact status choice value in your application before saving or testing either version.

6. Compare the approaches using the same data. Both should return the same count, but the tuned query avoids iterating over every appointment in Deluge.

7. Review frequently filtered fields and their types. Use Date, Number/Currency and choice fields where appropriate instead of storing these values as free text.

8. Open the Appointment Booking Form Report and dashboard. Apply filters and display only the fields needed for each report or chart. Use a limited range for list-style fetches and avoid fetching data inside loops.

9. Test with sample records and record the result. Do not report a measured speed improvement unless you actually time or benchmark both approaches.

5. Deluge / Configuration Code

A. Untuned loop-based approach

```
// Untuned approach: fetch all appointments and count in a loop
allAppointments = Appointment_Booking_Form[ID != 0];
pendingCount = 0;

for each appt in allAppointments
{
    if(appt.Appointment_Status == "Pending")
    {
        pendingCount = pendingCount + 1;
    }
}
info pendingCount;
```

B. Tuned query approach

```
// Tuned approach: filter and count in the query
pendingCount = Appointment_Booking_Form
    [Appointment_Status == "Pending"].count();

info pendingCount;
```

C. Limited-range fetch

```
// Tuned fetch: filter first and retrieve only a limited range
pendingAppointments = Appointment_Booking_Form
    [Appointment_Status == "Pending"] range from 1 to 50;

// Reuse pendingAppointments instead of fetching again inside a loop
```

Code note: Appointment_Booking_Form and Appointment_Status are example link names based on the project's form and status-board context. Before running the script, confirm the exact form and field link names in Zoho Creator. If the status field has a different link name or choice values, replace them accordingly. Use the code in a Deluge function or workflow action appropriate to the return type and context; `info` displays a debug value and does not itself create a dashboard card.

6. Implementation Observations

The supplied screenshots show the Appointment Booking Form and an Appointment Booking Form Report with five sample records. The dashboard design contains summary cards and charts, some of which display placeholders such as Sum(), Avg() and Ct(). The actual Auto-Calculate Net Fee workflow screenshot shows Deluge code reading consultation_fee and insurance_coverage_percent and checking for null values. These screenshots support that the app already uses forms, reports, dashboard components and Deluge; they do not show a completed performance benchmark or a saved optimized counting function.

7. Expected Output

For the same dataset, the loop-based and query-based approaches should return the same number of pending appointments. The query-based approach is expected to be more scalable because filtering and counting are handled by the Creator query instead of looping through every record in Deluge. Actual execution and timing must be verified in the live app.

8. Implementation Screenshots

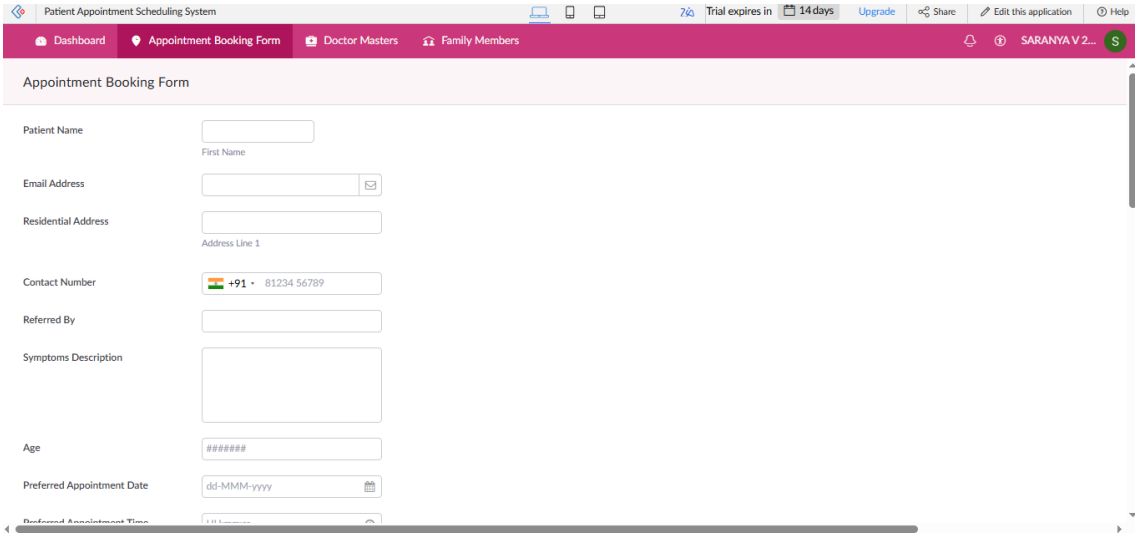


Figure 1: Appointment Booking Form with patient and appointment fields.

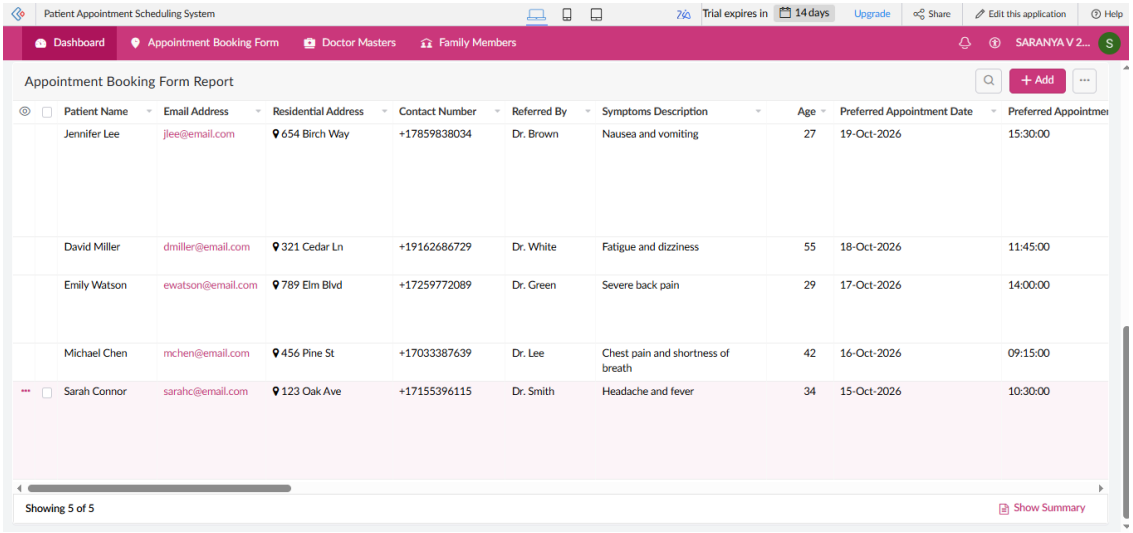


Figure 2: Appointment Booking Form Report showing saved sample appointment records.

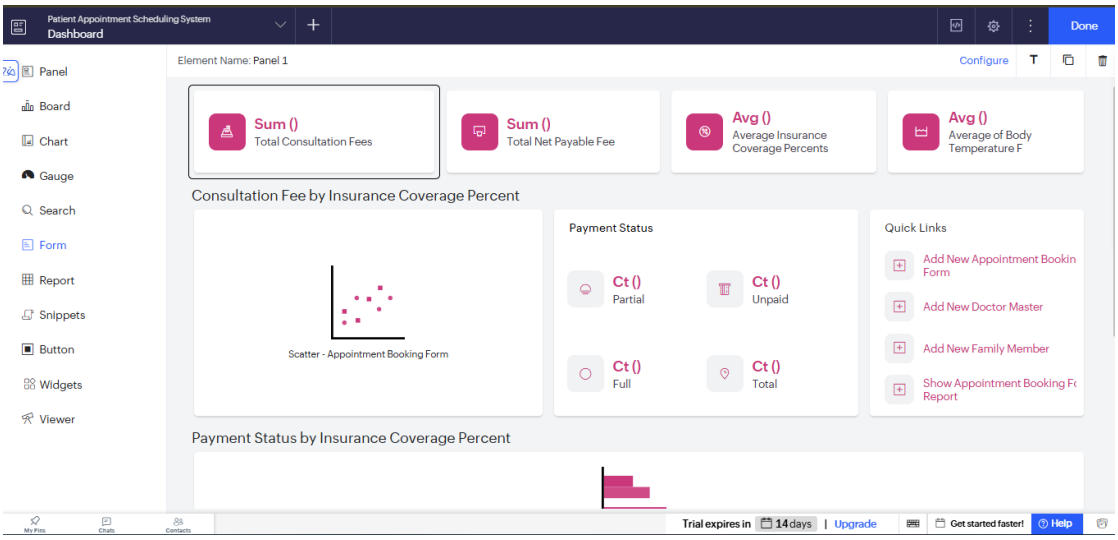


Figure 3: Dashboard design containing summary cards, payment-status panel, quick links and chart components.

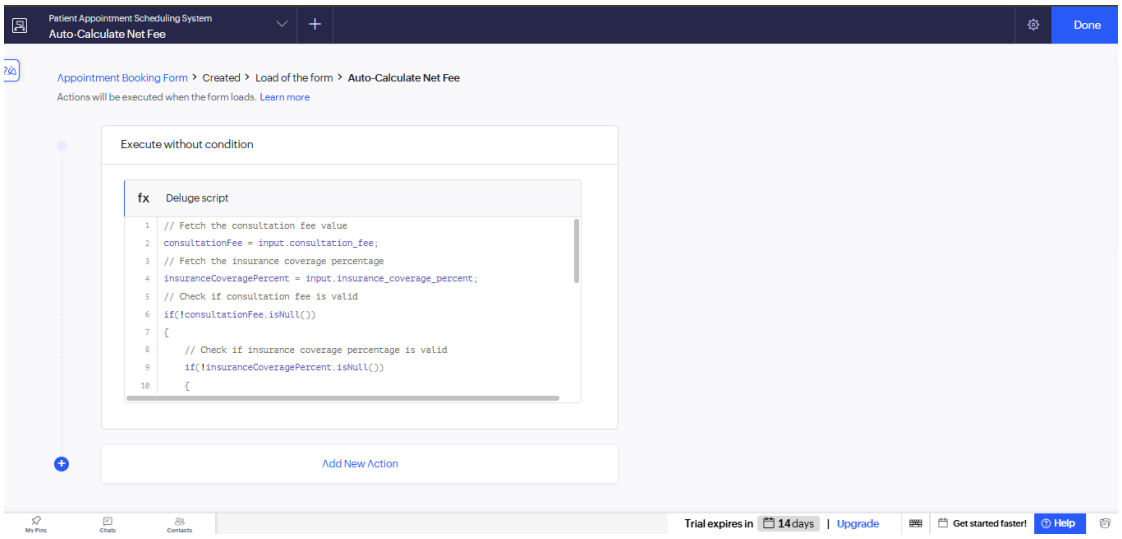


Figure 4: Existing Auto-Calculate Net Fee workflow using Deluge in the user's application.

9. Result

Performance optimization techniques were adapted to the Patient Appointment Scheduling System. The experiment documents typed-field choices, appropriate subform use, filtered queries, limited-range fetches and a reusable Deluge function pattern. The provided screenshots confirm the existing app structure and fee-calculation workflow; the proposed pending-appointment count should be tested with the actual field link names before claiming it is implemented.

10. Learning Outcome

Learnt how to reduce unnecessary record retrieval, use query criteria and count() instead of loop-based counting, choose appropriate field types for filtering, and keep reusable Deluge logic in functions. Also learnt to distinguish an optimization design from a measured performance result.